# Paradieschen Kistengenerator ; Vollständige Projektplanung & Entwicklungs-Workflow

# 🥬 PARADIESCHEN KISTENGENERATOR

## Vollständige Projektplanung & Entwicklungs-Workflow

### Multi-Agenten Vibe-Coding mit Claude Opus 4.6 + Grok

---

**Projekt:** Automatisierte Sortimentsgenerierung für Obst- & Gemüsekisten

**Auftraggeber:** Paradieschen Bio-Lieferdienst

**Live-Shop:** [shop.paradieschen.de](https://shop.paradieschen.de/obst-gemuesekisten)

**Methode:** Vibe-Coding — Zwei KI-Agenten coden, ein Mensch steuert

**Dokument-Typ:** Projektplanung & Entwicklungsanleitung

---

# TEIL 1: DAS PRODUKT

## Was bauen wir?

Eine App, die **auf Knopfdruck** fertige Obst- und Gemüsekisten zusammenstellt — preisoptimiert, mengengerecht und basierend auf Erfahrungswerten vergangener Kisten.

### Vorher vs. Nachher

```
HEUTE (manuell):                    MIT APP (automatisch):

Mitarbeiter rechnet stundenlang    →    1 Klick → Fertige Kiste
Excel-Listen, Kopfrechnen          →    Algorithmus optimiert Preis
Wissen nur in Köpfen               →    System lernt aus Vergangenheit
Fehleranfällig                     →    Mathematisch geprüft
```

---

## Die Kernidee — So funktioniert der Generator

> Direkt abgeleitet aus der E-Mail des Firmenbesitzers

### Der Generator denkt in 4 Bausteinen

```
┌──────────────────────┐  ┌──────────────────────┐  ┌──────────────────────┐
┌──────────────────────┐
│  MASTERPLAN  │  │  QUELLE  │  │ TAUSCHMUSTER │  │  PREISE  │
│              │  │          │  │              │  │          │
│ "Das Rezept" │  │"Die Zutaten" │  │"Die Portion" │  │"Das Budget" │
│              │  │ der Woche │  │  (Range)  │  │          │
│ OG12 =       │  │ KW24:     │  │ Zucchini: │  │ Zucchini: │
│ 3× Gemüse    │  │ Zucchini  │  │ 550–750 g │  │ 2,90 €/kg │
│ 1× Rohkost   │  │ Paprika   │  │ Paprika:  │  │ Paprika:  │
│ 1× Salat     │  │ Fenchel   │  │ 200–300 g │  │ 4,50 €/kg │
│ 3× Obst      │  │ Äpfel...  │  │ Fenchel:  │  │ ...       │
│              │  │           │  │ 600–800 g │  │           │
└──────────────┬──────────────┘  └──────────────┬──────────────┘  └──────────────┬──────────────┘
└──────────────┬──────────────┘
```

```
└──────────────────────────┴──────────────────────────┴──────────────────────────
──────┘
                    │
                    ┌──────────▼──────────┐
                    │ GENERATOR    │
                    │ ALGORITHMUS  │
                    └──────────┬──────────┘
                    │
                  ┌──────────────▼──────────────┐
                  │ FERTIGE KISTE       │
                  │ 8 Artikel, 14,39 €  │
                  │ → Freigabe → Shop   │
                  └──────────────────────────────┘
```

---

## Der Algorithmus — Der korrigierte Ablauf

> **Wichtigste Erkenntnis aus der Validierung:**
> Der Chef will **keinen** Generator der von Null rechnet.
> Er will einen Generator der **aus der Vergangenheit lernt**
> und diese Erfahrung auf die aktuelle Woche überträgt.

### Ablauf in der richtigen Reihenfolge

```
SCHRITT 1:  Mitarbeiter plant die QUELLE der Woche
            "Diese Woche gibt es: Zucchini, Paprika, Fenchel,
             Radieschen, Gurke, Äpfel, Birnen, Bananen"
```

SCHRITT 2: System sucht HISTORISCHEN MATCH ← HAUPTWEG

Vergleicht aktuelle Quelle mit allen

vergangenen Kisten und findet die ähnlichste

SCHRITT 3: Besten Treffer VERWENDEN

z.B. KW20/2024 hatte 5 von 8 gleiche Artikel

→ Übernimm dessen Mengen-Verteilung

SCHRITT 4: Nicht-passende Artikel ERSETZEN

durch Artikel aus der aktuellen Quelle

SCHRITT 5: Aktuelle PREISE einsetzen und neu RECHNEN

SCHRITT 6: RANGE-FEINTUNING wenn Preis nicht passt

Mengen innerhalb der Tauschmuster-Ranges anpassen

SCHRITT 7: Scheitert das? → NÄCHSTEN historischen Treffer nehmen

Scheitert alles? → Manueller Eingriff

```

### Warum diese Reihenfolge?

| Unser erster Entwurf (falsch) | Was der Chef wirklich meint (richtig) |

|:--|:--|

| Starte mit leerem Masterplan, fülle mit Quelle, optimiere Preis, nutze Historie nur als Notlösung | **Starte mit historischem Match**, nutze Erfahrung als Basis, mache Range-Feintuning nur zum Nachschärfen |

| Historie = Fallback am Ende | **Historie = Intelligenz am Anfang** |

| Statische Masterpläne | Masterpläne werden **auch aus Historie gelernt** |

---
```

## Die Datenquellen

### Was das Paradieschen liefert

| Datenquelle | Beschreibung | Status |
|:--|:--|:--:|
| **Artikelstamm** | Alle Artikel mit ID, Name, Gewicht, Einheit | ✅ Vorhanden (aus Screenshots) |
| **Masterpläne** | Kistentypen mit Slot-Struktur (OG12, OG15...) | ✅ Vorhanden (aus Screenshots) |
| **Tauschmuster** | Mengen-Ranges pro Artikel, Größe und Sortimentsart | ✅ Teilweise (Ranges müssen definiert werden) |
| **Historische Kisten** | Vergangene Zusammenstellungen | ⏳ Aus Fab-DB exportieren |
| **Preise** | Werden wöchentlich manuell gepflegt | ✅ Prozess existiert |

### Bis die Fab-DB-Daten da sind

> Wir arbeiten mit **realistischen Testdaten** basierend auf den vorhandenen Screenshots.

> Der Prototyp ist sofort lauffähig. Echte Daten werden später eingespeist.

---

# TEIL 2: DIE TECHNIK

## Datenmodell

### Alle Tabellen auf einen Blick

```
┌──────────────────────────┐      ┌──────────────────────────┐
┌──────────────────────────────┐
│   MASTERPLAN    │      │ MASTERPLAN_SLOT │      │   ARTIKEL_STAMM    │
```

```
│──────────────────────────│      │──────────────────────────│
│──────────────────────────────│
│ masterplan_id PK │──1:n─▶│ slot_id PK      │      │ artikel_id PK       │
│ name (OG12...)  │      │ masterplan_id FK │      │ sid                 │
│ groesse         │      │ kategorie        │      │ name                │
│ zielpreis_min   │      │ slot_nummer      │      │ kategorie           │
│ zielpreis_max   │      │ ist_pflicht      │      │ einheit             │
└──────────────────────────┘      └──────────────────────────┘      │ status              │
                                  └──────────────┬───────────────────┘
                                                 │
        ┌──────────────────────────┐   ┌──────────────────────────────┐   │
        │  TAUSCHMUSTER            │   │  PREIS_PFLEGE                │   │
        │──────────────────────────│   │──────────────────────────────│   │
        │ artikel_id FK      │◀──────│ artikel_id FK        │◀──┘
        │ groesse            │   │ preis_pro_einheit    │
        │ sortimentsart  NEU │   │ gueltig_ab / _bis    │
        │ min_menge          │   └──────────────────────────────┘
        │ max_menge          │
        └──────────────────────────┘


┌──────────────────────────┐   ┌──────────────────────────────────────┐
│  WOCHEN_QUELLE           │   │  HISTORISCHE_SORTIMENTE              │
│──────────────────────────│   │──────────────────────────────────────│
│ kalenderwoche      │   │ masterplan_id FK          │
│ jahr               │   │ kalenderwoche / jahr      │
│ slot_bezeichnung   │   │ artikel_zuweisungen JSON  │
│ artikel_id FK      │   │ mengen_zuweisungen JSON   │
└──────────────────────────┘   │ gesamtpreis               │
                               └──────────────────────────────────────┘


┌──────────────────────────────────────┐
┌──────────────────────────────────────┐
```

```
│  GELERNTE_MASTERPLAENE  NEU  │     │  GENERIERTE_KISTEN          │
│──────────────────────────────────────────────────────────────────│
│──────────────────────────────────────────────────────────────────│
│ basis_masterplan_id FK       │     │ masterplan_id FK            │
│ quell_historie_id FK         │     │ kalenderwoche / jahr        │
│ artikel_muster JSON          │     │ inhalt JSON                 │
│ mengen_muster JSON           │     │ gesamtpreis                 │
│ haeufigkeit                  │     │ optimierung_versuche        │
│ durchschnittspreis           │     │ match_score                 │
│                              │     │ match_quelle (hist. Ref.)   │
│                              │     │ status                      │
└──────────────────────────────────────────────────────────────────┘
└──────────────────────────────────────────────────────────────────┘
```

### Die Kernformel

Für jede Kiste berechnet das System:

$$P_{\text{gesamt}} = \sum_{i=1}^{n} (m_i \times p_i)$$

**Bedingung:** $P_{\min} \leq P_{\text{gesamt}} \leq P_{\max}$

**Constraint:** $m_{\min,i} \leq m_i \leq m_{\max,i}$ für jeden Artikel $i$

---

## Workflow-Diagramm des Generators

```mermaid
flowchart TD
    A["🟢 START\nMitarbeiter plant\nQuelle der Woche"] --> B["📦 QUELLE STEHT\nZucchini, Paprika,\nFenchel, Äpfel..."]
```

B --> C["🔍 HISTORISCHEN MATCH SUCHEN\nVergleiche Quelle mit allen\nvergangenen Kisten"]

C --> D{"Match\ngefunden?\n≥ 3 von 8\nArtikel gleich"}

D -->|"✅ JA"| E["📋 BESTEN TREFFER NEHMEN\nz.B. KW20/2024\n5 Übereinstimmungen"]

D -->|"❌ NEIN"| F["📋 STATISCHEN MASTERPLAN\nals Basis nutzen\n(OG12 etc.)"]

E --> G["🔄 ARTIKEL ERSETZEN\nNicht-übereinstimmende\ndurch aktuelle Quelle"]

F --> G

G --> H["💰 AKTUELLE PREISE\nEINSETZEN\nΣ Menge × Preis"]

H --> I{"Preis im\nZielrahmen?"}

I -->|"✅ JA"| J["📊 KISTE FERTIG\nStatus: Entwurf"]

I -->|"❌ NEIN"| K["⚖️ RANGE-FEINTUNING\nMengen in Tauschmuster-\nRanges anpassen"]

K --> L{"Passt nach\n≤ 10 Versuchen?"}

L -->|"✅ JA"| J

L -->|"❌ NEIN"| M["🔄 NÄCHSTBESTEN\nhistorischen Treffer\nnehmen"]

M --> N{"Noch Treffer\nvorhanden?"}

N -->|"JA"| G

N -->|"NEIN"| O["⚠️ MANUELL"]

J --> P["👤 Mitarbeiter\nprüft & gibt frei"]

P --> Q["🟢 Online im Shop\n+ In Historie gespeichert"]
```

---

## API-Endpunkte

| Endpunkt | Methode | Beschreibung |
|:--|:--:|:--|
| `/api/kiste/generieren` | POST | Neue Kiste generieren |
| `/api/kiste/{id}` | GET | Generierte Kiste abrufen |
| `/api/kiste/{id}/freigeben` | PUT | Kiste freigeben für Shop |
| `/api/masterplan` | GET | Alle Masterpläne listen |
| `/api/artikel` | GET | Alle Artikel (Filter: Kategorie) |
| `/api/quelle/{kw}/{jahr}` | GET/POST | Wochenquelle lesen/planen |
| `/api/preise` | GET/POST | Preise lesen/pflegen |
| `/api/historie` | GET | Historische Sortimente |
| `/api/muster/lernen` | POST | **NEU:** Muster aus Historie generieren |
| `/api/muster/match` | POST | **NEU:** Besten Match zur Quelle finden |

---

# TEIL 3: DIE ENTWICKLUNG

## Das Entwicklungsteam

### Drei Rollen, klare Aufgaben
```

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                                                                                                │
│                                                                                                                │
│                                                                                                                │
```

### Warum zwei LLMs?

| Aspekt | Ein LLM allein | Claude + Grok |
|:--|:--|:--|
| Geschwindigkeit | Sequentiell | **Parallel** — Frontend + Backend gleichzeitig |
| Fehlertoleranz | Blind für eigene Fehler | **Cross-Check** möglich |
| Spezialisierung | Generalist | Claude = Logik-Experte, Grok = UI-Experte |

---

## Der Phasenplan

### Übersicht

```
WOCHE 1                 WOCHE 2                 WOCHE 3
├─ Phase 1: DB          ├─ Phase 4: API         ├─ Phase 7: Zusammenführen
├─ Phase 2: Muster-Lernen  ├─ Phase 5: UI Preis     ├─ Phase 8: Testen
├─ Phase 3: Generator     ├─ Phase 6: UI Generator  ├─ Phase 9: Fab-DB Import
│                    │                    │
│  ═══ CLAUDE ═══════════    │  ═══ CLAUDE ═══         │  ═══ CLAUDE ═══════
│  ──────────────────────────    │  ─── GROK ═══════════    │  ─── GROK ═══════
│                    │    (parallel!)       │  ─── BEIDE ═══════
```

| Phase | Was | Wer | Tage |
|:-----:|:----|:---:|:----:|
| **1** | Datenbank & Testdaten | Claude | 1–2 |
| **2** | Muster-Lernen aus Historie | Claude | 2–3 |
| **3** | Generator-Kernlogik + Optimierung | Claude | 2–3 |

| **4** | API-Endpunkte | Claude | 1–2 |

| **5** | Frontend: Preispflege & Quelle | Grok | 3–4 |

| **6** | Frontend: Generator-UI & Dashboard | Grok | 3–4 |

| **7** | Zusammenführen Frontend + Backend | Beide | 2–3 |

| **8** | Testen & Bugfixing | Beide | 2–3 |

| **9** | Echte Fab-DB-Daten einbinden | Claude | 1–2 |

| | **Gesamt** | | **~20–25 Tage** |

---

## Phase 1: Datenbank & Testdaten

**Agent:** Claude Opus 4.6

**Dauer:** 1–2 Tage

**Ziel:** Lauffähige Datenbank mit realistischen Paradieschen-Daten

### Prompt für Claude — Phase 1

```
Du bist der Backend-Architekt für den "Paradieschen Kistengenerator".
Bio-Lieferdienst, Obst- und Gemüsekisten, shop.paradieschen.de.

AUFGABE: Erstelle eine vollständige SQLite-Datenbank.

TABELLEN:

1. ARTIKEL_STAMM
   artikel_id (PK), sid (unique), name, kategorie (Gemüse/Obst/Rohkost/Salat),
   einheit (Kilogramm/Stück), status (aktiv/saisonende)

2. MASTERPLAN
   masterplan_id (PK), name (unique, z.B. "OG12"), beschreibung,
```

groesse (S/M/L), zielpreis_min, zielpreis_max, ist_aktiv

3. MASTERPLAN_SLOT

slot_id (PK), masterplan_id (FK), kategorie, slot_nummer, ist_pflicht

4. TAUSCHMUSTER

muster_id (PK), artikel_id (FK), groesse (S/M/L),

sortimentsart (OG/RE/OOG), min_menge, max_menge, standard_menge, einheit

5. WOCHEN_QUELLE

quelle_id (PK), kalenderwoche, jahr, slot_bezeichnung, artikel_id (FK)

6. PREIS_PFLEGE

preis_id (PK), artikel_id (FK), preis_pro_einheit, gueltig_ab, gueltig_bis

7. HISTORISCHE_SORTIMENTE

historie_id (PK), masterplan_id (FK), kalenderwoche, jahr,

artikel_zuweisungen (JSON), mengen_zuweisungen (JSON), gesamtpreis

8. GELERNTE_MASTERPLAENE

gelernter_mp_id (PK), basis_masterplan_id (FK), quell_historie_id (FK),

artikel_muster (JSON), mengen_muster (JSON), haeufigkeit, durchschnittspreis

9. GENERIERTE_KISTEN

kiste_id (PK), masterplan_id (FK), kalenderwoche, jahr, inhalt (JSON),

gesamtpreis, optimierung_versuche, match_score, match_quelle, status

TESTDATEN basierend auf echten Paradieschen-Artikeln:

- Gemüse: Zucchini (13), Paprika gelb (241), Fenchel (22), Dicke Bohnen (84),
  Gurke (23), Mangold (43), Rote Bete (20), Lauch (32), Möhren (11m),
  Pastinaken (200), Grünkohl (89), Blumenkohl (3011, 304), Rosenkohl (38, 382),
  Sellerie (412), Staudensellerie (42), Radieschen (18), Rettich (209)

- Rohkost: Kresse (197), Paprika grün (24), Sprossen (17041, 216)
- Kräuter: Petersilie glatt (201), Petersilie kraus (202), Petersilienwurzel (203)
- Obst: Äpfel (60), Birnen (66), Bananen (861), Orangen (1991),
  Trauben (65), Kiwi (87)

- 8 Masterpläne: OG12, OG12-G, OG12-O, OG15, OG18, OG18-G, OG21, OG21-G
  OG12 Slots: Gemüse 1,3,4 + Rohkost 1 + Salat 1 + Obst 1,2,3
- Tauschmuster für Größe S, Sortimentsart OG:
  Zucchini: min 0.55, max 0.75 | Paprika: min 0.20, max 0.30
  Fenchel: min 0.60, max 0.80 | Stück-Artikel: min 1, max 1
- Fiktive Preise für KW24/2025
- 20 historische Sortimente (KW1–KW20/2024, verschiedene Typen)

TECHNOLOGIE: Python mit SQLAlchemy ORM.
DATEIEN: models.py, database.py, seed_data.py, requirements.txt
Kommentare auf Deutsch. Produktionsreif.
```

### Ergebnis prüfen

```
Frage dich: Kann ich die DB starten? Sehe ich Testdaten?
Test: python seed_data.py → Läuft ohne Fehler? ✅
```

---

## Phase 2: Muster-Lernen aus Historie

**Agent:** Claude Opus 4.6
**Dauer:** 2–3 Tage
**Ziel:** Algorithmus der aus vergangenen Kisten Muster extrahiert
```

> **Das ist NEU und die Kernintelligenz des Systems.**

> Der Chef schrieb: *"Wir lassen aus den bereits in der Vergangenheit geplanten Sortimenten Masterpläne generieren."*

### Prompt für Claude — Phase 2

```
Die Datenbank steht (models.py und seed_data.py sind fertig).

AUFGABE: Erstelle ein Modul "muster_lernen.py" mit folgender Logik:

FUNKTION 1: extrahiere_muster_aus_historie()
- Lade alle historischen Sortimente aus der DB
- Für jedes historische Sortiment:
  • Extrahiere das Muster: Welche Artikel-KATEGORIEN in welchen Slots
    (z.B. [Gemüse, Gemüse, Gemüse, Rohkost, Salat, Obst, Obst, Obst])
  • Extrahiere die Mengen-Verteilung relativ zum Gesamtpreis
  • Speichere als "gelernter Masterplan" in der Tabelle gelernte_masterplaene
- Erkenne wiederkehrende Muster und zähle deren Häufigkeit
- Berechne Durchschnittspreise pro Muster

FUNKTION 2: finde_besten_match(aktuelle_quelle, sortiment_typ)
- Input: Liste der aktuell verfügbaren Artikel + gewünschter Typ (z.B. OG12)
- Lade alle gelernten Masterpläne für diesen Typ
- Vergleiche die aktuellen Artikel mit jedem gelernten Masterplan:
  • Zähle Übereinstimmungen (gleicher Artikel oder gleiche Kategorie im Slot)
  • Berechne einen Match-Score:
    Score = Anzahl Übereinstimmungen / Anzahl Slots
- Sortiere nach Score (beste zuerst)
- Gib die Top-5 Matches zurück, jeweils mit:
  • Match-Score
```

- Welche Artikel übereinstimmen
- Welche Artikel ersetzt werden müssen
- Historische Referenz (KW/Jahr)

WICHTIG:
- Die Übereinstimmung soll auf SLOT-EBENE sein, nicht nur global
  (Gemüse 1 wird mit Gemüse 1 verglichen, nicht mit Obst 3)
- Bei der Suche: "5 Übereinstimmungen" bedeutet 5 von z.B. 8 Slots
  haben den gleichen oder einen ähnlichen Artikel

Erstelle auch test_muster_lernen.py zum Testen.
Nutze die SQLAlchemy-Modelle aus Phase 1.
Kommentare auf Deutsch.
```

### Ergebnis prüfen

```
Test: python test_muster_lernen.py
→ Werden Muster aus den 20 Test-Historien extrahiert?
→ Findet finde_besten_match() einen sinnvollen Treffer?
→ Ist der Match-Score nachvollziehbar (z.B. 5/8 = 0.625)?
```

---

## Phase 3: Generator-Kernlogik + Optimierung

**Agent:** Claude Opus 4.6
**Dauer:** 2–3 Tage
**Ziel:** Der vollständige Generator-Algorithmus in der richtigen Reihenfolge
```

### Prompt für Claude — Phase 3

```
DB steht, Muster-Lernen funktioniert. Jetzt der Generator.

AUFGABE: Erstelle "generator.py" mit dem KORREKTEN Ablauf:

HAUPTFUNKTION: generiere_kiste(sortiment_typ, groesse, kw, jahr)

ABLAUF (WICHTIG — diese Reihenfolge!):

1. QUELLE LADEN
   Lade verfügbare Artikel für KW/Jahr aus wochen_quelle

2. HISTORISCHEN MATCH SUCHEN (= HAUPTWEG, nicht Fallback!)
   Rufe finde_besten_match() aus muster_lernen.py auf
   Ergebnis: Top-5 Matches mit Scores

3. BESTEN TREFFER VERWENDEN (wenn Match ≥ 40%)
   - Übernimm die Mengen-Verteilung vom historischen Treffer
   - Ersetze nicht-übereinstimmende Artikel durch aktuelle Quelle
   - Ordne Ersatz-Artikel den passenden Slots zu (gleiche Kategorie)

4. FALLBACK: STATISCHEN MASTERPLAN (wenn kein Match ≥ 40%)
   - Lade den statischen Masterplan (z.B. OG12)
   - Ordne Artikel aus der Quelle den Slots zu
   - Setze Mengen als Mittelwerte der Tauschmuster-Ranges

5. AKTUELLE PREISE EINSETZEN
   - Lade Preise aus preis_pflege (gültig für diese KW)
   - Berechne Gesamtpreis = Σ(Menge × Preis)
```

6. RANGE-FEINTUNING (wenn Preis nicht im Zielrahmen)

- Preis zu hoch: Reduziere Mengen bei teuren kg-Artikeln
  (innerhalb Tauschmuster-Range, Schrittgröße 0.05 kg)
- Preis zu niedrig: Erhöhe Mengen bei günstigen kg-Artikeln
- Stück-Artikel (Range 1-1) NICHT ändern!
- Max 10 Iterationen

7. BEI MISSERFOLG: NÄCHSTEN TREFFER NEHMEN

- Wenn nach 10 Iterationen kein gültiger Preis:
  Nimm den nächstbesten historischen Match (Nr. 2, 3, 4, 5)
  Wiederhole Schritte 3–6
- Wenn alle 5 Treffer scheitern:
  Return {"status": "manuell_erforderlich", "grund": "..."}

8. ERGEBNIS SPEICHERN

- Speichere in generierte_kisten mit:
  inhalt, gesamtpreis, optimierung_versuche,
  match_score, match_quelle (hist. Referenz), status="entwurf"

ZUSÄTZLICHE FUNKTION: kiste_freigeben(kiste_id)

- Setzt Status auf "freigegeben"
- Speichert die freigegebene Kiste AUCH als neues historisches Sortiment
  (damit der Generator in Zukunft davon lernen kann!)

Erstelle test_generator.py mit diesen Szenarien:

- TEST 1: Happy Path (guter Match, Preis passt sofort)
- TEST 2: Feintuning nötig (Preis knapp daneben, Range-Anpassung hilft)
- TEST 3: Nächster Treffer nötig (erster Match scheitert preislich)
- TEST 4: Kein Match (Fallback auf statischen Masterplan)
- TEST 5: Gar nichts klappt (manueller Eingriff)

Nutze models.py und muster_lernen.py. Kommentare auf Deutsch.

```

### Ergebnis prüfen

```
Test: python test_generator.py
→ Alle 5 Szenarien bestanden?
→ Kommt bei TEST 1 eine fertige Kiste mit Preis im Rahmen?
→ Wird bei TEST 5 "manuell_erforderlich" zurückgegeben?
→ Wird nach Freigabe die Kiste in die Historie übernommen?
```

---

## Phase 4: API-Endpunkte

**Agent:** Claude Opus 4.6
**Dauer:** 1–2 Tage
**Ziel:** Alles als Web-Service verfügbar machen

### Prompt für Claude — Phase 4

```
Backend-Logik komplett. Jetzt die API.

AUFGABE: Erstelle FastAPI-Anwendung.

DATEIEN:
1. main.py — FastAPI App mit allen Routen
2. schemas.py — Pydantic-Modelle für Request/Response

ENDPUNKTE:
```
```

Generator:

POST /api/kiste/generieren

Input: {"typ": "OG12", "groesse": "S", "kw": 24, "jahr": 2025}

Output: Komplettes Kisten-JSON mit Positionen, Preisen, Match-Info

GET /api/kiste/{kiste_id}

PUT /api/kiste/{kiste_id}/freigeben

→ Kiste freigeben + automatisch in Historie übernehmen

Stammdaten:

GET /api/masterplan

GET /api/masterplan/{id}

GET /api/artikel?kategorie=Gemüse

Wöchentliche Pflege:

GET /api/quelle/{kw}/{jahr}

POST /api/quelle/{kw}/{jahr}

GET /api/preise?kw=24&jahr=2025

POST /api/preise

PUT /api/preise/{preis_id}

Muster & Historie:

GET /api/historie

POST /api/muster/lernen → Triggert Muster-Extraktion

POST /api/muster/match → Findet besten Match für Quelle

Konfiguration:

CORS erlauben für localhost

Swagger-Docs unter /docs

Server auf http://localhost:8000

Aktualisierte requirements.txt.

Kommentare auf Deutsch.

```

### Ergebnis prüfen

```

Start: uvicorn main:app --reload

Browser: http://localhost:8000/docs

→ Alle Endpunkte sichtbar?

→ POST /api/kiste/generieren mit Testdaten → Kiste kommt zurück?

```

---

## Phase 5: Frontend — Preispflege & Wochenquelle

**Agent:** Grok

**Dauer:** 3–4 Tage

**Ziel:** Admin-Oberfläche für die wöchentlichen Mitarbeiter-Aufgaben

> **⚡ Ab hier parallel zu Phase 3–4!**

> Grok kann am Frontend arbeiten, während Claude noch am Backend ist.

### Prompt für Grok — Phase 5

```

Ich baue das Frontend für den "Paradieschen Kistengenerator".

Ein Bio-Lieferdienst für Obst- und Gemüsekisten.

Backend: FastAPI auf http://localhost:8000

AUFGABE: Erstelle das Admin-Frontend.

SEITE 1 — PREISPFLEGE:

- Wochenauswahl oben (KW + Jahr Dropdown)
- Tabelle aller Artikel (GET /api/artikel)
- Spalten: Artikel-ID, Name, Kategorie, Einheit, Preis/Einheit, Status
- Inline-Editing: Klick auf Preis → Eingabe → Enter → Speichern
  (POST /api/preise)
- Filter-Dropdown: Alle / Gemüse / Obst / Rohkost / Salat
- Suchfeld für Artikelname
- Fortschrittsbalken: "18/32 Artikel gepflegt"
- Gelbes ⚠️ bei fehlenden Preisen
- Grünes ✅ bei gepflegten Preisen
- Deadline-Hinweis: "Preise müssen bis [Datum] stehen"

SEITE 2 — WOCHENQUELLE PLANEN:

- Wochenauswahl oben
- Links: Verfügbare Artikel (aus GET /api/artikel, nach Kategorie gruppiert)
- Rechts: Die Slots aus dem Masterplan
  (Gemüse 1, Gemüse 2, Gemüse 3, Rohkost 1, Salat 1, Obst 1, Obst 2, Obst 3)
- Drag-and-Drop: Artikel von links nach rechts ziehen
- ODER: Dropdown pro Slot
- Leere Slots rot markiert
- Speichern → POST /api/quelle/{kw}/{jahr}
- "Alle Slots gefüllt ✅" Anzeige

DESIGN:

- Clean, modern, professionell
- Farbschema: Grün-Töne (Bio-Feeling, passend zu Paradieschen)
- Responsive (auch auf Tablet)
- Deutsche Beschriftungen überall
- Font: Gut lesbar, nicht zu verspielt

TECHNOLOGIE: React oder Vue.js (was du für besser hältst)

DATEIEN: Alle Komponenten, api.js Service, package.json, Startanleitung

```

---

## Phase 6: Frontend — Generator-UI & Dashboard

**Agent:** Grok

**Dauer:** 3–4 Tage

**Ziel:** Die Hauptfunktion der App — Kisten generieren und freigeben

### Prompt für Grok — Phase 6

```

Frontend-Grundgerüst steht. Jetzt die Hauptseiten.

SEITE 3 — KISTEN-GENERATOR (die wichtigste Seite!):

Oberer Bereich — Eingabe:
  - Dropdown: Sortiment-Typ (OG12, OG15, OG18...)
    Daten: GET /api/masterplan
  - Toggle-Buttons: Größe S / M / L
  - Wochenauswahl: KW + Jahr
  - Großer grüner Button: "🔄 KISTE GENERIEREN"
    → POST /api/kiste/generieren

Unterer Bereich — Ergebnis (nach Klick):

  Info-Box oben:
  - Gesamtpreis GROSS anzeigen (z.B. "14,39 €")
  - Grüner Hintergrund wenn im Rahmen, Roter wenn nicht
```
```

- Zielrahmen als Text: "Rahmen: 10–15 €"
- Match-Info: "Basiert auf KW20/2024 · 5/8 Übereinstimmungen"
  (Das ist wichtig! Der Nutzer soll sehen WOHER die Kiste kommt)
- Methode: "Historischer Match + Range-Feintuning, 3 Versuche"

Artikel-Liste (gruppiert nach Kategorie):

Jede Gruppe hat eine Überschrift (GEMÜSE, ROHKOST, SALAT, OBST)

Jeder Artikel zeigt:
- Emoji + Name (z.B. 🥒 Zucchini)
- Aktuelle Menge (z.B. "0,65 kg")
- SLIDER der die Range zeigt:
  Linker Anschlag = Minimum (z.B. 0,55)
  Rechter Anschlag = Maximum (z.B. 0,75)
  Punkt = aktuelle Menge
  Bei Stück-Artikeln: Slider deaktiviert (fix)
- Positionspreis rechts (z.B. "1,89 €")

LIVE-UPDATE: Wenn Slider bewegt wird:
→ Positionspreis neu berechnen
→ Gesamtpreis oben aktualisieren
→ Farbe der Info-Box anpassen (grün/rot)

Drei Buttons ganz unten:
[✏️ Anpassen] → Slider werden editierbar
[✅ Freigeben] → PUT /api/kiste/{id}/freigeben
[🗑️ Verwerfen] → Kiste löschen

SEITE 4 — DASHBOARD:
- Tabelle aller generierten Kisten
- Spalten: KW, Typ, Größe, Preis, Match-Score, Status, Datum

- Filter: Status (Entwurf/Freigegeben), Typ
- Klick auf Zeile → Öffnet Seite 3 mit dieser Kiste
- Statistik oben: "12 Kisten diese Woche, 8 freigegeben"

SEITE 5 — HISTORIE:
- Vergangene Kisten durchblättern
- Nützlich um zu sehen worauf der Generator basiert

NAVIGATION (Sidebar oder Top):
📊 Dashboard
🔄 Generator
💰 Preispflege
📦 Wochenquelle
📜 Historie

Gleicher Stil wie Phase 5.
```

---

## Phase 7: Zusammenführung

**Agent:** Beide (Claude führt, Grok unterstützt)
**Dauer:** 2–3 Tage
**Ziel:** Frontend und Backend verbinden

### Prompt für Claude — Phase 7

```
Hier ist der aktuelle Stand:

BACKEND (von dir erstellt):

[Hier main.py, models.py, generator.py, muster_lernen.py einfügen]

FRONTEND (von Grok erstellt):

[Hier api.js und Hauptkomponenten einfügen]

AUFGABE:

1. Prüfe ob alle API-Aufrufe im Frontend zu den Backend-Endpunkten passen
2. Prüfe ob die JSON-Strukturen (Request + Response) übereinstimmen
3. Liste ALLE Unstimmigkeiten auf (Feldnamen, Datentypen, URLs)
4. Erstelle Fixes für beide Seiten
5. Erstelle docker-compose.yml:
   - Backend: Python FastAPI Container
   - Frontend: Node/React Container
   - Datenbank: SQLite Volume
6. Erstelle .env mit allen Konfigurationen
7. Erstelle README.md mit Schritt-für-Schritt-Startanleitung:
   "Wie starte ich die App als Nicht-Programmierer?"

```

---

## Phase 8: Testen

**Agent:** Claude Opus 4.6

**Dauer:** 2–3 Tage

**Ziel:** Alles funktioniert End-to-End

### Prompt für Claude — Phase 8

```

App läuft. Jetzt gründlich testen.

AUFGABE: Erstelle test_integration.py mit diesen Szenarien:

TEST 1 — HAPPY PATH:

Preise gepflegt + Quelle geplant + OG12 generieren

→ Erwartung: Kiste mit 8 Positionen, Preis 10–15 €, Match-Score > 0

TEST 2 — HISTORISCHER MATCH DOMINANT:

Quelle hat 6 von 8 Artikeln gleich wie KW10/2024

→ Erwartung: Hoher Match-Score (≥ 0.75), Mengen ähnlich wie historisch

TEST 3 — RANGE-FEINTUNING:

Setze alle Preise 50% höher als normal

→ Erwartung: Algorithmus reduziert Mengen, trotzdem im Rahmen

TEST 4 — TREFFER-KASKADE:

Setze Preise so, dass erster Match scheitert

→ Erwartung: System nimmt automatisch nächsten Match

TEST 5 — KEIN MATCH:

Quelle mit komplett unbekannten Artikeln

→ Erwartung: Fallback auf statischen Masterplan

TEST 6 — MANUELLER EINGRIFF:

Preise extrem hoch (10×)

→ Erwartung: Status "manuell_erforderlich"

TEST 7 — FEHLENDE DATEN:

Keine Preise für KW24 gepflegt

→ Erwartung: Fehlermeldung "Preise nicht gepflegt für KW24"

TEST 8 — FREIGABE + LERNEFFEKT:

Kiste generieren → Freigeben → Erneut generieren

→ Erwartung: Freigegebene Kiste taucht als historischer Match auf

Erstelle auch einen HTML-Report der die Testergebnisse schön darstellt.
```

---

## Phase 9: Echte Daten

**Agent:** Claude Opus 4.6

**Dauer:** 1–2 Tage

**Ziel:** Fab-DB-Daten ersetzen Testdaten

### Prompt für Claude — Phase 9 (wenn Daten da sind)

```
Die Fab-DB-Daten liegen als CSV/Excel vor.

AUFGABE: Erstelle import_fabdb.py:

1. CSV/Excel einlesen (pandas)
2. Mapping: Fab-DB Spalten → Unsere Tabellen
   [HIER SPALTENÜBERSCHRIFTEN DER FAB-DB EINFÜGEN]
3. Testdaten löschen, echte Daten einfügen
4. Validierung: Pflichtfelder prüfen, Duplikate erkennen
5. AUTOMATISCH Muster-Lernen triggern nach Import
6. Report: "142 Artikel importiert, 520 historische Kisten,
   37 Muster gelernt"

Backup-Skript das vorher die DB sichert.
```
```

---

# TEIL 4: DER KOMMUNIKATIONSFLUSS

## Woche für Woche

### Woche 1 — Das Fundament

```
TAG 1 (Montag)
━━━
```

"Alle 5 Testszenarien bestanden?" ✅ / ❌

TAG 5 (Freitag)

---

Du → Claude: Phase 4 Prompt (API) + alle Dateien

Du → Grok: Phase 5 Prompt (Frontend starten!) ← PARALLEL!

```

### Woche 2 — Die Oberfläche

```

TAG 6 (Montag)

---

Claude → Du: main.py, schemas.py (API fertig)

Du: uvicorn main:app --reload

http://localhost:8000/docs öffnen

"Alle Endpunkte sichtbar? Kiste generierbar?" ✅

Grok → Du: Erste Frontend-Komponenten (Preis-Pflege)

Du: npm start

"Sehe ich die Preistabelle?" ✅ / ❌

TAG 7–8 (Dienstag–Mittwoch)

---

Du → Grok: Phase 6 Prompt (Generator-UI)

+ API-Response-Beispiel von Claude als Kontext

Grok → Du: Generator-Seite mit Slidern, Dashboard

TAG 9–10 (Donnerstag–Freitag)

---

Du: Frontend-Feinschliff mit Grok

Erste Verbindungsversuche Frontend ↔ Backend

```

### Woche 3 — Zusammenführung

```
TAG 11–12 (Montag–Dienstag)
━━━

## Die 5 Regeln für den Vibe-Coder

> **Regel 1: IMMER KONTEXT MITGEBEN**
> Bei jedem neuen Prompt: Die relevanten Dateien aus vorherigen Phasen mitsenden.
> LLMs haben kein Gedächtnis zwischen Sessions.
> Beispiel: Phase 3 Prompt → Sende models.py und muster_lernen.py mit.

> **Regel 2: EINE PHASE NACH DER ANDEREN**
> Nie Phase 3 starten wenn Phase 2 nicht funktioniert.
> Jede Phase MUSS getestet und bestätigt sein bevor es weitergeht.

> **Regel 3: FEHLERMELDUNGEN SOFORT WEITERLEITEN**
> Fehler kopieren (komplett!) und mit dem gesamten Code an das zuständige LLM:
> *"Dieser Fehler tritt auf bei [Datei/Zeile]. Hier der Code: [...]. Behebe den Fehler."*

> **Regel 4: FRONTEND UND BACKEND PARALLEL**
> Ab Woche 2 kann Grok am Frontend arbeiten während Claude noch an der API ist.
> Das spart eine ganze Woche.

> **Regel 5: PHASE 7 BRAUCHT GEDULD**
> Die Zusammenführung wird 2–3 Runden brauchen. Das ist normal.
> Jedes Mal wenn etwas nicht passt: Code von BEIDEN Seiten an Claude senden.

---

## Was am Ende da steht

### Die Ordnerstruktur

```
paradieschen-kistengenerator/
│
```

```
├── backend/                    ← Von Claude erstellt
│   ├── main.py                 ← FastAPI Server
│   ├── models.py               ← Datenbank-Tabellen
│   ├── database.py             ← DB-Verbindung
│   ├── muster_lernen.py        ← NEU: Muster aus Historie lernen
│   ├── generator.py            ← Der Algorithmus (korrigierte Reihenfolge)
│   ├── schemas.py              ← API-Datenformate
│   ├── seed_data.py            ← Testdaten
│   ├── import_fabdb.py         ← Für echte Daten (Phase 9)
│   ├── requirements.txt
│   └── tests/
│       ├── test_muster_lernen.py
│       ├── test_generator.py
│       └── test_integration.py
│
├── frontend/                   ← Von Grok erstellt
│   ├── src/
│   │   ├── components/
│   │   │   ├── Dashboard.jsx
│   │   │   ├── Generator.jsx       ← Hauptscreen mit Slidern
│   │   │   ├── PreisPflege.jsx
│   │   │   ├── WochenQuelle.jsx    ← Drag-and-Drop
│   │   │   ├── KistenVorschau.jsx
│   │   │   └── Historie.jsx
│   │   ├── services/
│   │   │   └── api.js
│   │   ├── App.jsx
│   │   └── index.jsx
│   ├── package.json
│   └── README.md
│
├── docker-compose.yml          ← Von Claude (Phase 7)
```

```
├── .env
└── README.md                    ← Gesamtanleitung
```

### Was die App kann (Feature-Liste)

| Feature | Beschreibung | Phase |
|:--|:--|:--:|
| 🔄 Kisten generieren | Auf Knopfdruck, preisoptimiert | 3 |
| 🧠 Aus Vergangenheit lernen | Historische Kisten als Hauptbasis | 2 |
| ⚖️ Mengen intelligent anpassen | Range-Feintuning innerhalb Tauschmuster | 3 |
| 🔄 Automatische Treffer-Kaskade | Nächster Match wenn erster scheitert | 3 |
| 💰 Preispflege | Übersichtliche Tabelle mit Fortschritt | 5 |
| 📦 Wochenquelle planen | Drag-and-Drop Slots | 5 |
| 📊 Live-Slider | Mengen manuell nachjustieren mit Preis-Update | 6 |
| 🏷️ Match-Transparenz | Zeigt woher die Kiste stammt (KW/Jahr) | 6 |
| 📈 Dashboard | Übersicht aller Kisten mit Status | 6 |
| 📜 Historie durchblättern | Vergangene Kisten einsehen | 6 |
| ✅ Freigabe-Workflow | Prüfen → Anpassen → Freigeben → Online | 4+6 |
| 🔄 Selbstlernendes System | Freigegebene Kisten fließen in Historie zurück | 3 |
| 📥 Fab-DB Import | Echte Daten einlesen wenn verfügbar | 9 |

---

# TEIL 5: ANHANG

## Rechenbeispiel zur Demonstration

### OG12, Größe S, KW 24/2025

**Schritt 1 — Quelle der Woche:**

Zucchini, Paprika gelb, Fenchel, Radieschen, Gurke, Äpfel, Birnen, Bananen

**Schritt 2 — Historischer Match gefunden:**

KW20/2024 hatte: Zucchini, Paprika gelb, **Mangold**, Radieschen, Gurke, Äpfel, Birnen, **Orangen**

→ **6 von 8 Übereinstimmungen** → Match-Score: 0,75

**Schritt 3 — Mengen vom Match übernehmen, Artikel ersetzen:**

| Slot | Historisch (KW20/24) | Aktuell (KW24/25) | Menge (übernommen) | Preis |
|:--|:--|:--|:--:|:--:|
| Gemüse 1 | Zucchini ✅ | Zucchini | 0,65 kg | 1,89 € |
| Gemüse 3 | Paprika gelb ✅ | Paprika gelb | 0,25 kg | 1,13 € |
| Gemüse 4 | ~~Mangold~~ | **Fenchel** (ersetzt) | 0,70 kg | 2,24 € |
| Rohkost 1 | Radieschen ✅ | Radieschen | 1 Stk | 1,20 € |
| Salat 1 | Gurke ✅ | Gurke | 1 Stk | 0,90 € |
| Obst 1 | Äpfel ✅ | Äpfel | 1,00 kg | 3,00 € |
| Obst 2 | Birnen ✅ | Birnen | 0,65 kg | 2,47 € |
| Obst 3 | ~~Orangen~~ | **Bananen** (ersetzt) | 0,90 kg | 2,25 € |
| | | **Gesamt:** | | **15,07 €** |

**Schritt 4 — Zielrahmen 10–15 € → 15,07 € ist KNAPP DRÜBER:**

```
Feintuning Iteration 1: Birnen  0,65 → 0,55 kg  → -0,38 €
Feintuning Iteration 2: Äpfel   1,00 → 0,90 kg  → -0,30 €

Neuer Gesamtpreis: 14,39 € → IM RAHMEN ✅
```

**Ergebnis:**

```json
{
```

```
  "kiste_id": 42,
  "masterplan": "OG12",
  "groesse": "S",
  "kw": 24,
  "gesamtpreis": 14.39,
  "match_score": 0.75,
  "match_quelle": "KW20/2024",
  "optimierung_versuche": 2,
  "methode": "Historischer Match + Range-Feintuning",
  "status": "entwurf"
}
```

---

## Problemlösung — Wenn etwas nicht funktioniert

| Situation | Prompt an welches LLM |
|:--|:--|
| Python-Fehler im Backend | → Claude: "Fehler [X] in [Datei]. Hier der Code. Fix." |
| Frontend zeigt nichts an | → Grok: "Seite ist leer. Browser-Konsole zeigt: [Fehler]. Code: [...]" |
| API antwortet nicht | → Claude: "Endpunkt /api/kiste/generieren gibt 500. Logs: [...]" |
| Frontend ↔ Backend passt nicht | → Claude: "Hier Frontend api.js + Backend schemas.py. Was passt nicht?" |
| Generator findet keinen Preis | → Claude: "Alle 5 Matches scheitern. Preise: [...]. Ranges: [...]" |
| Neuer Kistentyp gebraucht | → Claude: "Füge Masterplan RE15 hinzu. Slots: 5× Gemüse, 2× Rohkost..." |
| Fab-DB Daten endlich da | → Claude: Phase 9 Prompt + CSV-Dateien |

---

> **Zusammenfassung:**

> - **Produkt:** App die automatisch Paradieschen-Kisten generiert

> - **Kern-Intelligenz:** Lernt aus vergangenen Kisten (nicht von Null rechnen)

> - **Feintuning:** Mengen-Ranges + Preisoptimierung als Nachschärfung

> - **Methode:** Zwei LLMs (Claude = Backend, Grok = Frontend), du steuerst

> - **Timeline:** ~3 Wochen bis zum funktionierenden MVP

> - **Daten:** Sofort mit Testdaten starten, Fab-DB-Export parallel organisieren